MachBoost: Speculative Decoding from Local Context for On-Device LLMs

Vistrit Pandey
vistrit@blinkfire.com

July 2026

Abstract

Autoregressive decoding remains serial even when a model fits comfortably on a laptop. MachBoost tests a narrow way around that constraint: use tokens from explicit local context as speculative candidates, verify a block with the target model, and return to the runtime's native decoder when no candidate exists. The current implementation combines context-only n-gram lookup, block verification, in-place MLX KV-cache rewind, verifier continuation tokens, and native `mlx-lm` streaming. We compare it with native cached generation rather than a synchronous Python token loop. On an Apple M1 Max, using `mlx-community/Qwen2.5-3B-Instruct-4bit`, a 64-token code-continuation fixture reaches a 2.36× median paired wall-time speedup over three fresh-nonce repeats. All three accelerated token sequences match the native greedy outputs; the median run accepts 51 context tokens and reduces logical target forwards from 64 to 14. A retrieval answer and an open-ended control have no eligible initial continuation, take the native path, and measure 0.96× and 1.00× median paired speedup. These nine runs do not establish universal acceleration. They locate one useful operating region and expose why earlier measurements against stateless or per-token baselines overstated the gain.

1 Introduction

Running LLMs on a laptop has become realistic, but it has not made autoregressive decoding disappear. A small model on Apple Silicon may fit in memory and still feel slow because the loop is serial: produce a token, update the prefix, run the model again. That bottleneck is especially visible in developer workflows, where the model often sits inside an editor, command runner, notebook, or local assistant.

Speculative decoding offers a way around part of this loop. A cheap source guesses several future tokens; the target model checks the guess; accepted tokens are emitted without paying for one full target step each. Prior work usually gets the draft from another model, extra heads, or a reference sequence. MachBoost takes the reference idea and applies it to the local machine. If the user is asking about a repo, config, note, policy, or retrieved passage, the likely continuation may already be present in that local text.

The scope is deliberately modest. MachBoost does not try to accelerate every prompt. It tries to accelerate grounded prompts without changing the target model's greedy output. If the local text is unhelpful, the method can be slower; the implementation exposes that fact through measurement and falls back to the serial path.

This paper contributes the following:

- a context-only drafter and initial-candidate gate that avoid treating the prompt wrapper itself as external evidence;

- a cache-aware MLX verifier that fuses continuation and draft scoring, rewinds rejected cache entries in place, and resumes native decoding when useful context ends;
- a paired benchmark against native `mlx-lm` generation with raw rows, run-order alternation, environment provenance, and exact token comparisons;
- negative experiments with a small draft model, an MTP draft path, DFlash, an Apple Neural Engine draft, and lossless weight compression.

2 Background and Related Work

Speculative decoding was introduced as a way to accelerate autoregressive transformer inference without changing outputs by drafting candidate tokens and verifying them in parallel with the target model [6]. Speculative sampling independently developed a related draft-and-verify method for sampled decoding and reported 2–2.5× speedups in a distributed setting [4]. Subsequent work explored architecture-level or self-drafting variants. Medusa adds multiple decoding heads to predict future tokens and verifies candidate trees [3].

MachBoost is closest to methods that exploit references or prompt overlap rather than an auxiliary draft model. LLMA observes that outputs in retrieval and reference-grounded scenarios often share spans with the input reference, then verifies copied spans to preserve greedy output [13]. Prompt Lookup Decoding replaces the draft model with n-gram matching over the prompt [8]; PLD+ extends that idea by using language-model artifacts such as attention or hidden states to rank candidate spans [9]. Hugging Face Transformers exposes prompt lookup through `prompt_lookup_num_tokens` in assisted generation [5]. MachBoost differs mainly in packaging and operating assumptions: it treats local files and retrieved snippets as ordinary inputs to the accelerator, keeps benchmark data machine-readable, and makes low-overlap prompts a measured fallback case rather than an error.

Results above 2× elsewhere generally require more than process tuning. ReDrafter trains a recurrent draft conditioned on target hidden state and reports up to 2.3× on Apple Silicon [14]. QuantSpec changes the draft's weight and KV-cache representation and reports speedups up to about 2.5× in long-context inference [10]. Mirror-SD maps complementary draft and target work across GPU and NPU devices and reports 2.8–5.8× on server-scale models [2]. At the runtime layer, BaseRT's native Metal kernels report up to 1.35× over MLX decode rather than a universal 2× [7]. These are different operating points: trained model changes, heterogeneous execution, or low-level kernels. MachBoost's current context path needs none of them, but only applies when the requested continuation overlaps local text.

The same distinction appears outside text generation. Diffusion workloads can reuse intermediate transformations across denoising steps; EasyCache reports 2.1–3.3× for several video models [15]. That mechanism does not transfer directly to autoregressive text decoding. It does, however, support a broader product design in which one package selects a backend-specific accelerator rather than pretending that a single algorithm fits text, image, audio, and video models.

The implementation targets common local inference stacks. Hugging Face Transformers provides the broad model API surface for Python inference [11]. MLX is an Apple Silicon-oriented array framework and runtime used here for Mac-native evaluation [1]. Our measurements use a Qwen-family model converted for MLX; Qwen3 is the relevant public model-family report [12].

3 Problem Setting

Let M be a target autoregressive model and let x be the prompt token sequence. Greedy decoding emits

$$y_t = \arg\max_v M(v \mid x, y_{<t})$$

for each generation step t. MachBoost receives a corpus C derived from prompt text, retrieved context, local files, or user-provided strings. The goal is to reduce wall-clock latency while returning the same sequence y that greedy decoding with M would return.

This paper stays with greedy decoding. Sampling-compatible speculative decoding is possible in principle, but greedy argmax verification gives a simple correctness check: the accelerated sequence must match the serial baseline token-for-token.

4 Method

The implementation has a drafter, a verifier, and an escape hatch to native generation. The escape hatch is important: speculative decoding is a conditional optimization, not a replacement decoder.

4.1 Drafting from Nearby Text

The drafter indexes token n-grams from a corpus C. At generation time it looks at the current prefix $p = x \| \hat{y}$, searches for suffixes of p that occur in C, and proposes the tokens that followed the best matching span. Longer suffix matches are preferred, with draft length as a secondary signal. Only explicit context is indexed in the reported adaptive mode. An earlier implementation indexed the concatenation of prompt and context; because benchmark prompts already displayed the source passage, that design could draft from the prompt wrapper and blur the boundary between available context and generated history.

Before entering the verifier loop, MachBoost asks whether the generation boundary has a context candidate. If not, it calls native `mlx-lm` generation immediately. This inexpensive gate catches open-ended prompts and the retrieval fixture in the current suite. It does not predict future candidates that might appear later in a response, so it favors low overhead over maximum recall.

4.2 Verification Against the Target Model

Given a prefix p and draft $d = (d_1, \ldots, d_k)$, the verifier checks how many draft tokens equal the target model's greedy continuation. A verifier-capable runtime scores a block in one target call and compares target argmax tokens at the relevant positions. If the first a draft tokens are accepted, they are committed. On a mismatch, the target's residual token is emitted. When the whole block is accepted, the verifier also returns the next target token already present in the logits. The following round scores that pending token together with the next draft, avoiding a separate target call between accepted blocks.

Under deterministic target arithmetic, the intended invariant is:

$$\text{Boost}(M, x, C, T) = \text{Greedy}(M, x, T)$$

for a maximum generation length T. Batched and scalar floating-point paths can still choose different argmax tokens at a near tie, so this is tested rather than assumed. Every benchmark row records an `output_match` flag from the native and accelerated token IDs.

4.3 Keeping the Cache Path Stable

Verification writes candidate entries to the MLX prompt cache. Accepted entries remain useful; rejected entries must not survive. MachBoost trims the rejected suffix in place when the cache type permits it. If safe trimming is unavailable, the service reconstructs the accepted prefix. Once no further context candidate exists, the service hands the live verifier cache to `mlx_lm.generate_step`; the remaining response therefore uses the runtime's optimized asynchronous decoder instead of a Python-level `argmax`/synchronize loop.

4.4 Deciding When to Use the Drafter

Drafting from local text is sometimes the wrong choice. Open-ended prompts often accept only a few draft tokens, so verification overhead dominates. MachBoost therefore exposes a benchmark step. For a prompt family, it measures:

- token match between baseline and boosted output,
- wall-clock speedup,
- accepted draft tokens divided by generated tokens,
- target-call or forward-call reduction.

The high-level API can also calibrate an accelerator across representative prompts:

```
from machboost import Accelerator, GatePolicy

boost = Accelerator.from_mlx(model, context_paths=["./docs", "./src"])
calibration = boost.calibrate(
    prompts,
    max_tokens=32,
    gate_policy=GatePolicy(min_speedup=1.05,
                           min_acceptance_rate=0.10),
)
text, stats = boost.generate(prompt, max_tokens=128)
```

If the measured workload fails the policy, `generate` uses the native backend path. Calibration and the per-request initial-candidate gate serve different purposes: calibration enables a workload family, while the gate prevents verifier overhead on an individual request with no draft at its boundary.

5 Implementation

MachBoost is implemented as a small Python package with no required runtime dependencies. Optional extras install backend-specific dependencies:

- `machboost[mlx]` for MLX and `mlx-lm`,
- `machboost[hf]` for PyTorch and Transformers,
- Ollama HTTP support for benchmarking and capability detection.

The core package defines a `StepService` protocol with `next_token(prefix)` and an optional `verify(prefix, candidate)` hook. A service with only `next_token` can use the wrapper but cannot skip target work. A verifier can accept several draft tokens per target call. The high-level `Accelerator` loads MLX or Hugging Face models, reads strings, files, or directories as context, and exposes `benchmark`, `calibrate`, and `generate`. The Ollama HTTP adapter remains a wrapper: Ollama's public generation API does not expose the target token IDs, logits, mutable KV cache, and block-verification hook this implementation needs. Native integration would require a runner-level change rather than an HTTP option.

6 Evaluation

6.1 Machine and Model

Measurements were collected on an Apple M1 Max with 10 CPU cores and 32 GB unified memory, running macOS 26.5.2. The environment used Python 3.13.3, MLX 0.31.2, `mlx-lm` 0.31.3, and MachBoost 0.1.3. No thermal or performance warning was reported immediately before or after the run. The model was `mlx-community/Qwen2.5-3B-Instruct-4bit`.

The benchmark requests 64 greedy tokens for three fixture families: literal Python continuation, retrieval-grounded answering, and open-ended writing. Each family is repeated three times with a fresh nonce. Baseline and accelerated order alternate. Wall time includes prompt processing. The baseline calls `mlx-lm`'s native streaming generator; it is not MachBoost's `next_token` loop. The artifact `results/mlx_native_adaptive_qwen25_3b_20260713.json` contains prompts, token previews, raw times, package versions, hardware data, and each run's order.

6.2 Native-Baseline Results

Fixture/path	Match	Paired speedups	Median	Base tok/s	MB tok/s
Code/adaptive	100%	2.51, 2.36, 1.59	2.36×	77.14	127.30
RAG/native	100%	0.96, 1.00, 0.94	0.96×	91.18	88.47
Open/native	100%	1.00, 1.01, 0.96	1.00×	98.56	99.46

Table 1: Three fresh-nonce repeats per fixture. Speedup is paired wall time; the throughput columns are per-column medians and therefore do not necessarily reproduce the median paired ratio. "MB" denotes MachBoost.

The code fixture begins at a boundary that occurs in the supplied source. MachBoost accepts 51 of 64 output tokens in every repeat and records 14 logical target forwards instead of 64, a 78.1% reduction. The slowest accelerated repeat is still 1.59×, which also shows the variance of short local runs. The median paired speedup is 2.36×.

The retrieval answer is semantically grounded in the note but its generation boundary does not form an eligible context continuation. It therefore exercises native fallback. So does the open-ended control. Their paired ratios fluctuate around one; RAG's three-run median is 0.96×. This small overhead is measurement noise and wrapper setup, not an acceleration result. Across all nine rows, token IDs match their corresponding native outputs.

An aggregate median over the three fixture types would be close to 1× and would hide the conditional nature of the method. We report paths separately. For a user-facing system, the useful statistic is coverage together with conditional speedup: how often a request is eligible, and how much the eligible requests improve.

6.3 Why Earlier Ratios Were Rejected

Earlier repository artifacts reported 3–8× on MLX. Their baseline disabled the prompt cache or performed a synchronized target call for each token. That path ran near 10 tokens/s and was not representative of native `mlx-lm`, which uses cached asynchronous generation. Those artifacts remain available as implementation diagnostics, but they cannot support a production speed claim. Replacing that baseline was also what exposed a regression in MachBoost's serial tail; switching the tail to native generation restored parity for non-overlap prompts.

A one-repeat Hugging Face artifact likewise found MachBoost context lookup competitive with Transformers prompt lookup, but it predates the current paired harness. It is useful for designing a larger comparison, not for the headline result.

6.4 Exploratory Alternatives

We tested several ways to extend acceleration beyond literal context overlap. Table 2 summarizes the local probes. They were not all run under one benchmark protocol, so the numbers are diagnostic rather than comparative.

Path	Local result	Main constraint
DFlash, Qwen3.5 9B	0.66× at 128; 0.96× at 512	M1 lacks the newer accelerator path; draft emulation costs more than accepted work saves.
Qwen2.5 0.5B draft for 3B target	0.38–0.78×	Acceptance of roughly 36–58% did not repay serial draft generation.
Ollama embedded MTP	12.7–24.3 tok/s versus roughly 40 tok/s baseline	The tested draft depths added overhead on this model and machine.
ANE 0.5B draft	51.1 decode tok/s	A one-token ANE draft cannot feed a 3B MLX target near 102 tok/s economically; multi-token heads are needed.
Lossless Q4 compression	1.08–1.15× ideal entropy bound	The quantized weights are already close to high entropy; decoder cost would reduce the gain further.

Table 2: Exploratory paths that did not beat native generation locally. Each result is specific to the tested model, implementation, and M1 Max.

The failures share a simple accounting problem. A draft only helps when the time removed from target decoding exceeds draft generation, verification, synchronization, and cache repair. A smaller model is not automatically a cheap enough draft. Moving it to another processor is not sufficient either if it predicts one token at a time. The strongest published results solve this with trained multi-token prediction, high acceptance, or concurrent heterogeneous execution rather than with process priority.

7 Limitations

The native-baseline evidence has nine rows, one model, one quantization, one machine, and controlled 64-token prompts. The accelerated fixture is intentionally favorable: the desired code is substantially present in context. It demonstrates the mechanism but does not estimate how often real users encounter an eligible boundary. Longer repository sessions, multiple 3B–9B architectures, prefill-heavy prompts, and energy measurements remain untested under the corrected harness.

Only greedy decoding is evaluated. Exact token equality under the tested execution paths does not prove equality for sampling, beam search, or every floating-point near tie. Images, audio, and video are not accelerated by the current package. Their models need modality-specific adapters; applying text n-gram verification to them would be a category error. Ollama is also not accelerated through its public HTTP API.

Finally, this work does not introduce prompt lookup itself. The engineering contribution is the adaptive package path, MLX cache integration, native fallback, and reproducible baseline correction. A novelty claim beyond that would need broader comparisons with LLMA, Prompt Lookup Decoding, PLD+, and current trained speculative systems.

8 Next Experiments

There are two plausible tracks. The first keeps models unchanged: add an online eligibility predictor, a suffix index that can re-enter speculation after native tokens, and native verifier hooks for MLX and llama.cpp/Ollama. Evaluation should report request coverage, time to first token, decode throughput, peak memory, energy, and exact outputs against each backend's own optimized generator.

The second track accepts model-specific preparation. A multi-token drafter distilled from the target could run on the Apple Neural Engine while target verification runs on the GPU, following the heterogeneous lesson of Mirror-SD. Quantized draft weights and KV state could reduce memory traffic in long contexts. Both ideas exceed the scope of a wrapper and should be treated as separate research prototypes with quality and compatibility tests.

For image and video generation, the package can expose the same routing and evidence interface while using activation or transformation caching such as EasyCache. This would make MachBoost a common control plane for specialized accelerators, not a claim that one decoder optimization applies to every model.

9 Conclusion

MachBoost currently clears 2× in one defensible setting: literal context-backed code continuation on a 3B MLX model, measured against native cached generation. The three paired runs have a 2.36× median speedup and exact token agreement. Requests without an initial context candidate take the native path and remain near parity. That is narrower than the original ambition, but it is a sound base. The corrected result also makes the next problem precise: increase eligibility without paying for a slow serial draft, then validate the gain against optimized native runtimes across models and modalities.

References

[1] Apple Machine Learning Research. Mlx: An array framework for apple silicon. https://github.com/ml-explore/mlx, 2023. Software.

[2] Nikhil Bhendawade, Kumari Nishu, Arnav Kundu, Chris Bartels, Minsik Cho, and Irina Belousova. Mirror speculative decoding: Breaking the serial barrier in llm inference. Apple Machine Learning Research, 2025.

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

[5] Hugging Face. Assisted decoding. https://huggingface.co/docs/transformers/en/assisted_decoding, 2026. Accessed July 6, 2026.

[6] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 2023.

[7] Prabod Rathnayaka, Fabian Waschkowski, and Lukas Wesemann. BaseRT: Best-in-class LLM inference on apple silicon via native metal. *arXiv preprint arXiv:2607.00501*, 2026.

[8] Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023. GitHub repository.

[9] Shwetha Somasundaram, Anirudh Phukan, and Apoorv Saxena. Pld+: Accelerating llm inference by leveraging language model artifacts. *arXiv preprint arXiv:2412.01447*, 2024.

[10] Rishabh Tiwari, Haocheng Xi, Aditya Tomar, Coleman Hooper, Sehoon Kim, Maxwell Horton, Mahyar Najibi, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Quantspec: Self-speculative decoding with hierarchical quantized kv cache. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[11] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45. Association for Computational Linguistics, 2020.

[12] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[13] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models. *arXiv preprint arXiv:2304.04487*, 2023.

[14] Aonan Zhang, Ray Zhang, Yunfei Cheng, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024.

[15] Xin Zhou, Dingkang Liang, Kaijin Chen, Tianrui Feng, Xiwu Chen, Hongkai Lin, Yikang Ding, Feiyang Tan, Hengshuang Zhao, and Xiang Bai. Less is enough: Training-free video diffusion acceleration via runtime-adaptive caching. *arXiv preprint arXiv:2507.02860*, 2025.